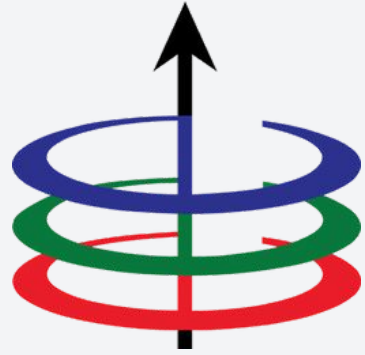




Lecture 1.1

Basic C++ Programming Elements and Libraries

EEE 121 THVW1

Steven S. Sison

- C++ Fundamentals
- Flow Control
- Functions
- Arrays and Strings

- **C++ Fundamentals**
- Flow Control
- Functions
- Arrays and Strings

C++ vs Python - Hello world!

Python

```
print("Hello world!")
```

C++

```
#include <iostream>
using namespace std;

int main(void){
    cout << "Hello world!" << endl;
}
```

Another comparison - Solving for GCD

Python

```
1 def gcd(u, v):
2     # we will use Euclid's algorithm
3     # for computing the GCD
4     while v != 0:
5         r = u % v    # compute remainder
6         u = v
7         v = r
8     return u
9
10 if __name__ == '__main__':
11     a = int(raw_input('First value: '))
12     b = int(raw_input('Second value: '))
13     print 'gcd: ', gcd(a,b)
```

Python and C++ implementation of a program that takes 2 integers, u and v (where $u \geq v$), and ***prints the greatest common divisor of the two integers***

C++

```
1 #include <iostream>
2 using namespace std;
3
4 int gcd(int u, int v) {
5     /* We will use Euclid's algorithm
6        for computing the GCD */
7     int r;
8     while (v != 0) {
9         r = u % v;    // compute remainder
10        u = v;
11        v = r;
12    }
13    return u;
14 }
15
16 int main( ) {
17     int a, b;
18     cout << "First value: ";
19     cin >> a;
20     cout << "Second value: ";
21     cin >> b;
22     cout << "gcd: " << gcd(a,b) << endl;
23     return 0;
24 }
```


C++ Libraries

- We don't have to write all the functions we need. We can reuse functions and classes from a collection called libraries
- C++ libraries are similar to Python modules
- Format:
 - Using libraries built-in with the compiler: `#include <library_name>`
 - Using own libraries: `#include "library_name" // looks at current directory`
- Some standard libraries that will be useful to us:

▪ <code>iostream</code>	▪ <code>string</code>	▪ <code>set</code>
▪ <code>stdlib</code>	▪ <code>cmath</code>	▪ <code>iomanip</code>
▪ <code>stdio</code>	▪ <code>queue</code>	▪ <code>ctime</code>
▪ <code>algorithm</code>	▪ <code>stack</code>	▪ <code>utility</code>
▪ <code>vector</code>	▪ <code>map</code>	

Namespaces

- Provides a space to define or declare a variable, method, or classes.
- Namespaces are used to organize code in a logical manner in order to prevent conflicts between variables, functions, and objects.

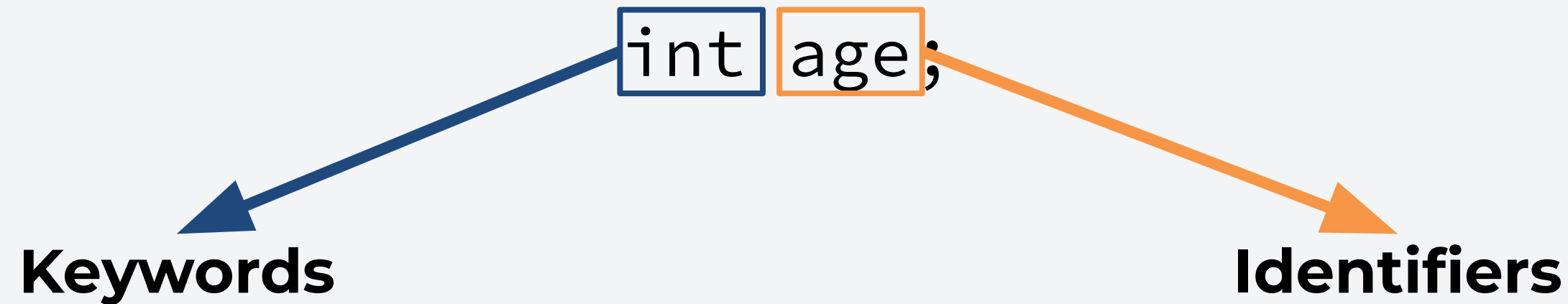
```
4 namespace first{
5     void func(){
6         cout << "First Space" << endl;
7     }
8 }
9
10 namespace second{
11     void func(){
12         cout << "Second Space" << endl;
13     }
14 }
15
16 using namespace first;
17
18 int main(){
19     func();    // Outputs "First Space"
20     return 0;
21 }
```

Problem: We have two definition for func(). How does the compiler know which definition to use?

Solution: Specify the namespace to be used

Notice how we also use “using namespace std” at the start of our C++ code? :)

Keywords and Identifiers



- Predefined words that are used by the compiler.
- **Example:** Too many to mention, but you can find them [here](#).
- Unique names that are given to variables, classes, functions by the programmer.

Variables and Constants

- Variables are used as a storage to contain data.
 - In python, we are not required to declare variables before using it.
 - In C++, we are **required** to declare the variable first and specify its type before using it. Although we can choose to initialize a value or not.

```
int q;           // Variable declaration
int r = 0;       // Declaration with initial value
int s = 5, t = 10; // Multiple declaration
```

- We can also make the variable immutable or a **constant** by adding const before the datatype during declaration.

```
const int u = 15; // Constant variable u with value 15
```

Primitive Data Types

Data Type	Meaning	Size	Example Literals
int	integer	2/4 bytes (16/32-bit)	38
float	floating-point	4 bytes (32-bit)	3.14f
double	floating-point	8 bytes (64-bit)	3.14
char	single character	1 byte (8-bit)	'e'
wchar_t	wide character	2 bytes (16-bit)	L'e'
bool	boolean	1 byte (8-bit)	TRUE, FALSE
void	empty	0	None

Literals are data used to represent fixed values (but they are not constant values). **Example:** 1, 2, 'a', etc.

Type Modifiers

Type modifiers are used to modify the properties of primitive data types. There are four type modifiers:

Type Modifier	Meaning	Sample Usage
short	Used for storing small integers (-32767 to 32767)	<code>short a = 121</code>
long	Used for storing large integers (-2147483647 to 2147483647)	<code>long b = 121123138</code> <code>long double c = 0.121121L</code>
signed	Can store positive and negative integers	<code>signed int d = -121</code>
unsigned	Can only store non-negative integers	<code>unsigned int f = 121</code>

Data Type Conversion

Recall: C++ uses static typing compared to the dynamic typing of python.

- As such, **implicit type casting** might occur during variable assignment.

```
int a = 5;           // type(a) = int
double b;            // type(b) = double
b = a;               // Converts 5 to 5.00 since type(b) is a double

int c;               // type(c) = int
char d = 'a';        // type(d) = char
c = d;               // Converts d to an integer. Question: What is the value of 'a'?
```

- However, we can also perform **explicit type casting** whenever necessary.

```
int e = 4, f = 3;
double c;
c = 4/3;              // c = 1.0 (4/3 = 1.33 -> 1 typecasted to double -> 1.0)
c = double(4)/3;      // c = 1.33 (4/3 = 1.33 since 4 is a double)
```

Basic Input and Output

- In python, we use the `print()` command to display formatted text into a screen.
- In C++, we use `cout` along with the `<<` operator to display an output.
 - Additionally, we can also add multiple `<<` operator to display text.
 - We can also use the `endl` common to create a new line.

```
#include <iostream>
using namespace std;

int main() {
    // prints the string enclosed in double quotes
    cout << "This is C++ Programming";
    return 0;
}
```


Basic Input and Output

- In python, we use the `input()` command to accept user input.
 - Note that in python, the `input()` command **always** converts the input into a string.
- In C++, we can use the `cin` command together with the `<<` operator to accept user input.
 - Note that **we are required** to initialize a variable first that will accept our input.
 - We also have to keep in mind the type of the variable that will store our input.

Basic Input and Output

```
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter an integer: ";
    cin >> num;    // Taking input
    cout << "The number is: " << num;
    return 0;
}
```

Arithmetic Operators

Operator	Description
+	Addition: Used to add two numbers or concatenate strings.
-	Subtraction: Used to subtract two numbers.
*	Multiplication: Used to multiply two numbers.
/	Division: Used to divide two numbers.
%	Modulo: Used to perform the modulo operation.

Bitwise Operators

Operator	Description
&	Binary AND
	Binary OR
^	Binary XOR
~	Binary one's complement
<<	Bitwise Shift Left
>>	Bitwise Shift Right

Assignment Operators

Operator	Example	Equivalent
=	a = 1	a = 1
+=	a += 1	a = a + 1
-=	a -= 1	a = a - 1
*=	a *= 1	a = a * 1
/=	a /= 1	a = a / 1
%=	a %= 1	a = a % 1

Note. This can also be extended to bitwise operators.

- C++ Fundamentals
- **Flow Control**
- Functions
- Arrays and Strings

Relational Operators

- These operators are used to check the relationship between operands.

Operator	Meaning	Equivalent
==	Is equal to	121 == 123 gives false
!=	Is not equal	121 != 123 gives true
>	Greater than	121 > 123 gives false
<	Less than	121 < 123 gives true
>=	Greater than or equal to	121 >= 121 gives true
<=	Less than or equal to	121 <= 121 gives true

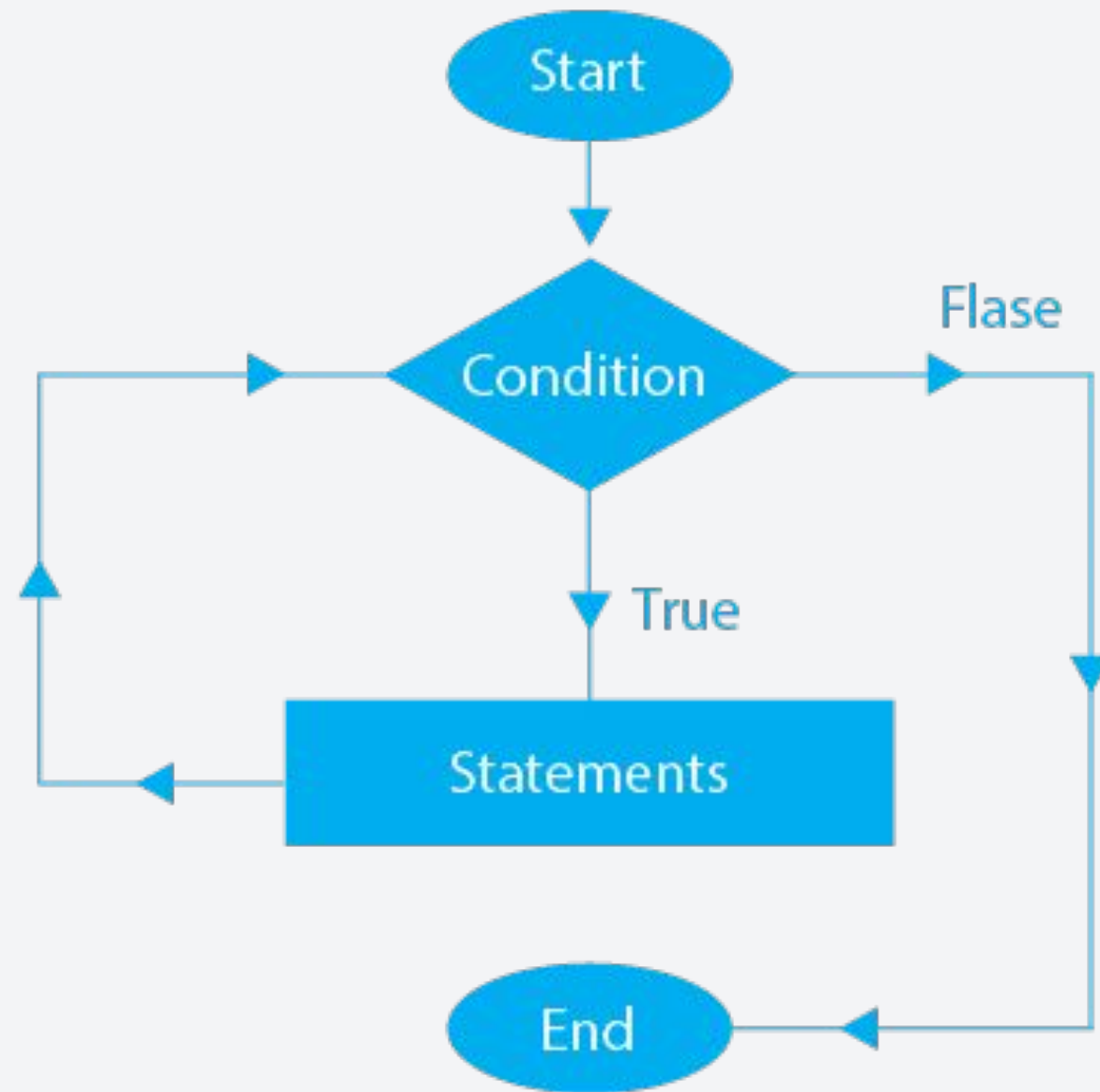
Logical Operators

- These operators are used to check whether a relationship is true or false.

Operator	Example	Equivalent
&&	expression1 && expression2	Logical AND True only if all operands are true
	expression1 expression2	Logical OR True if at least one operand is true
!	!expression1	Logical NOT True only if the operand is false

Looping (while, do-while, for loop)

While Loop

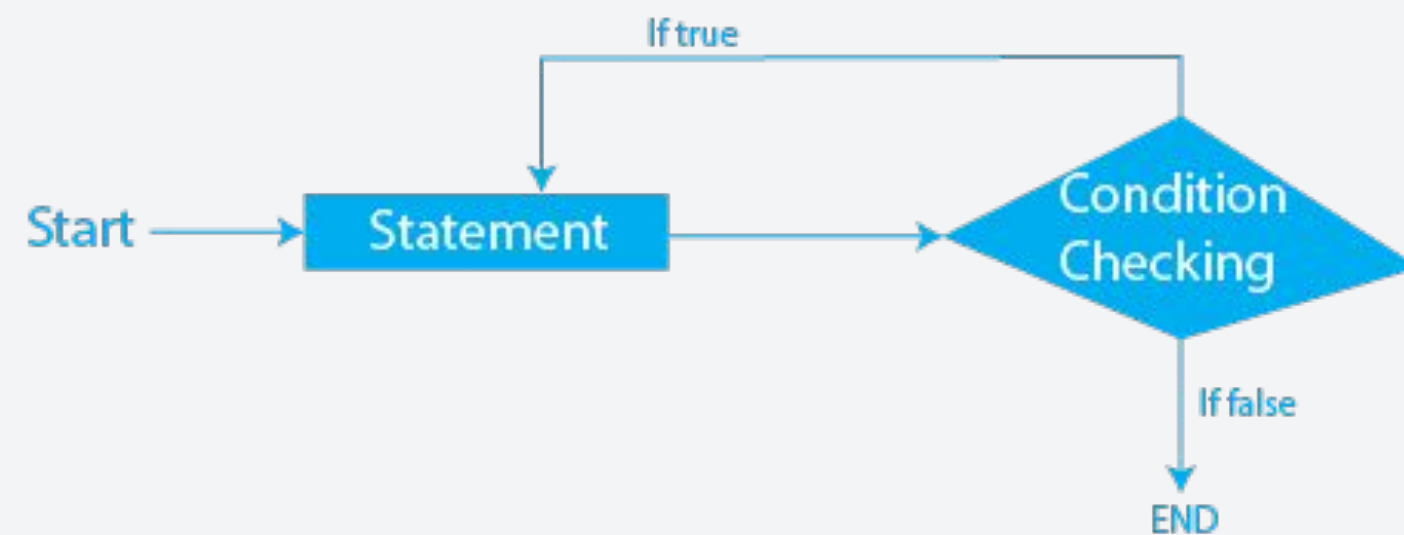


```
int i = 0;
while (i <= 100){
    i++;
}
```

While the **condition** is true, execute the **block of code**.

Looping (while, do-while, for loop)

Do-While

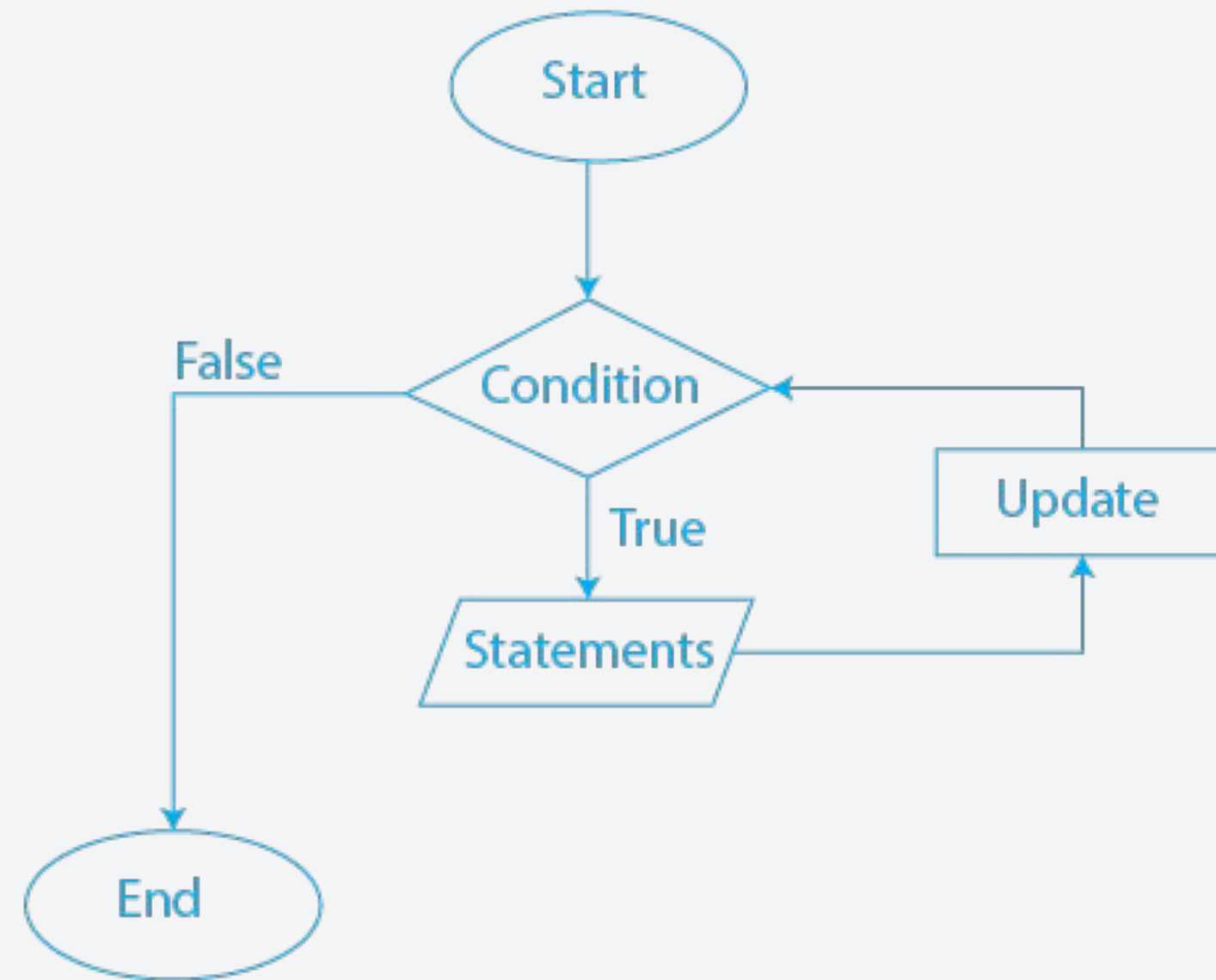


```
int i = 0;
do{
    i++;
}
while (i <= 100);
```

Execute the block of code. If the **condition** is still true, execute again.

Looping (while, do-while, for loop)

For Loop



```
for (int i = 0; i <= 100; i++) {  
    // Execute code here  
}
```

Do **initialization** and **check if the condition** is true. If true, **execute the code** and **update**. Repeat.

Conditionals (If-else, switch case)

If-else

```
if (statement1){  
    // Execute code block 1  
} elif (statement2) {  
    // Execute code block 2  
} else {  
    // Execute code block 3  
}
```

- Useful for executing code that has multiple expressions/conditions.

Switch-case

```
switch (expression){  
    case constant1:  
        // Execute code for constant1  
        break;  
    case constant2:  
        // Execute code for constant2  
        break;  
    default:  
        // Execute code if no match for any constant  
}
```

- Useful for single expressions that can have multiple values.

- C++ Fundamentals
- Flow Control
- **Functions**
- Arrays and Strings

Creating and Declaring a Function

Let's say you want to create a function that adds two numbers, you must follow the syntax:

```
returnType function_name(parameter1, parameter2){  
    // function body  
}
```



```
int add(int x, int y){  
    return x+y;  
}
```

- The return type can be any of the primitive data types.
- In python, you don't have to specify the data type of the parameters. In C++, you are required to specify.

Creating and Declaring a Function

- Functions should be declared before `main()`.

Method 1

```
int add(int x, int y){  
    return x+y;  
}  
  
int main(){  
    int x = 1, y = 2;  
    add(x,y);  
}
```

Method 2

```
int add(int x, int y);    // This is also a valid declaration  
  
int main(){  
    int x = 1, y = 2;  
    add(x,y);  
}  
  
int add(int x, int y){  
    return x+y;  
}
```


- C++ Fundamentals
- Flow Control
- Functions
- **Arrays and Strings**

Array Initialization and Usage

Let's say you want to create an integer array containing 5 elements, you need to follow the given syntax:

```
dataType array_name[size];           // Use this if you want an empty array
dataType array_name[size] = [1,2,3,4,5] // Use this if you want to place the elements

int numbers[5] = [1,2,3,4,5]          // Sample initialized array
```

- In C++, you are required to specify the type of elements you are going to put inside the array. You can't put more than one type.
- Arrays have **fixed size**. You can no longer modify it after initialization.
- You don't need to know the all of the elements of the array. You can initialize it with empty elements.

Array Initialization and Usage

Array elements can be accessed through its index (**NOTE:** C++ starts at 0 index).

```
int numbers[5] = [1,2,3,4,5]           // Sample initialized array

// You can change the value of an element through the assignment operator
numbers[3] = 1           // [1,2,3,1,5]

// You can print the element of an array
cout << numbers[1]      // Output: 2

// If you exceed the array's length, an error will be raised
numbers[10]             // Array out-of-bounds
```


One-dimensional vs Multidimensional Arrays

- In C++ (even in python), you are allowed to create arrays with multiple dimensions. Think of it as an array inside an array.
- These are particularly useful for representing data that is inherently 2D, 3D, and more such as a matrix (2D) and tensors (3D and above).

	Col1	Col2	Col3	Col4	
Row1	Arr[0][0]	Arr[0][1]	Arr[0][2]	Arr[0][3]	
Row2	Arr[1][0]	Arr[1][1]	Arr[1][2]	Arr[1][3]	
Row3	Arr[2][0]	Arr[2][1]	Arr[2][2]	Arr[2][3]	
Row4	Arr[3][0]	Arr[3][1]	Arr[3][2]	Arr[3][3]	
⋮					

```
int matrix[row_size][column_size];           // Initializes a 2D array

// You can access its elements the same way you do in an array
matrix[1][2] = 2    // Sets the element at row 1, column 2 as 2
```

Strings as char array vs Strings as an Object

String as Char Array

- In C, strings are essentially just an **array of characters**.
- A string is defined as an array terminated by a null character “\0”

```
char str[4] = "C++"    // NOTE: The string requires a
                        // size of 4 because of the
                        // terminal character '\0'

// You can access its element like an array
str[2] = "C"           // str = "C+C"
```

String as an Object

- In C++, we can use the <string> library to define string as a class.
- It's still an array of characters but using the string class provides more functionality.

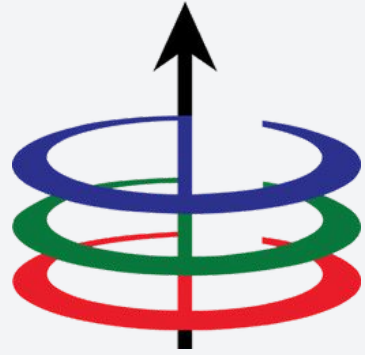
```
#include <string>

string str = "C++"

// Sample Methods
str.append(" is fun!") // str = "C++ is fun!"
str.find("+")          // Output: 2 (It finds the first
                        // occurrence of the character
```

- C++ Fundamentals
- Flow Control
- Functions
- Arrays and Strings

Any questions?



Lecture 1.1

Basic C++ Programming Elements and Libraries

EEE 121 THVW1

Steven S. Sison